

Labwork 9: Histogram Equalization

Pham Tien Nhat

October 4, 2026

1 Implementation

The Labwork 9 is implemented on the CPU version. 9a computes the histogram of a grayscale image, and 9b uses it to equalize the image.

```
import time
import numpy as np
import matplotlib.pyplot as plt

image = plt.imread('1.jpeg')
plt.imshow(image)
h, w, d = image.shape
```



Figure 1: Input image (407×634).

The input image is converted to gray with $(R + G + B)/3$. Each channel is cast to integer before the sum.

```
gray = np.zeros((h, w), np.uint8)
for y in range(h):
    for x in range(w):
        gray[y, x] = (int(image[y, x, 0]) + int(image[y, x, 1])
                      + int(image[y, x, 2])) / 3
```



Figure 2: Gray image, input of 9a and 9b.

9a: Histogram: The histogram is an array of 256 bins, where bin i counts the number of pixels having gray level i . Each pixel adds 1 to the bin of its gray level.

```
histo = np.zeros(256, np.int32)

start = time.time()
for y in range(h):
    for x in range(w):
        histo[gray[y, x]] += 1
print(f"CPU time: {time.time() - start} seconds")

print(f"Total pixels: {histo.sum()} = {h} x {w} = {h * w}")
```

The highest bin is gray level of around 70 with over 4000 pixels, where lowest areas are around 200 to 250.

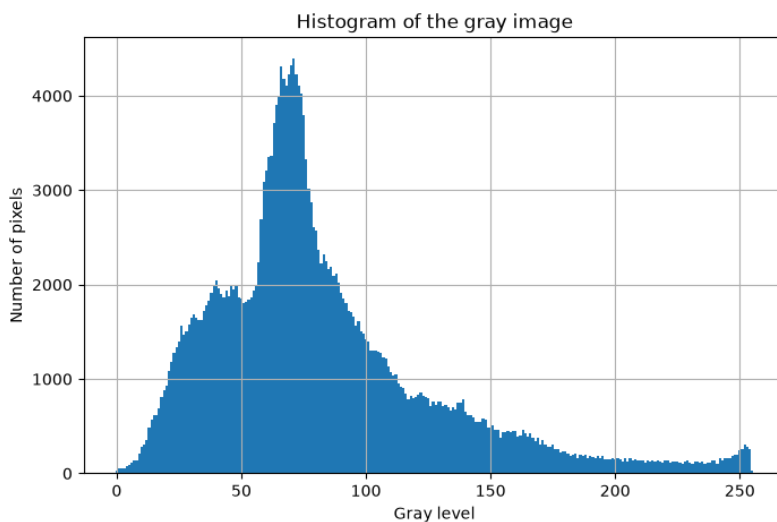


Figure 3: Histogram of the gray image.

9b: Histogram equalization: The intensities are recalculated with 4 steps including:

- Probability of each gray level;
- cumulative distribution function;

- scale $c_i \in [0..1]$ to $h_i \in [0..255]$ where $h_i = c_i \times 255$;
- each pixel of gray level i becomes h_i .

```
n = h * w

# probability of each gray level
p = np.zeros(256, np.float32)
for j in range(256):
    p[j] = histo[j] / n

# cumulative distribution function
c = np.zeros(256, np.float32)
c[0] = p[0]
for i in range(1, 256):
    c[i] = c[i - 1] + p[i]

# scale c from [0..1] to [0..255]
hmap = np.zeros(256, np.uint8)
for i in range(256):
    hmap[i] = c[i] * 255

# transform each pixel: gray level i becomes hmap[i]
equalized = np.zeros((h, w), np.uint8)
for y in range(h):
    for x in range(w):
        equalized[y, x] = hmap[gray[y, x]]
```

Figure below shows the difference in equalized and original image, where equalized image tends to show brighter contrast than the original gray one.

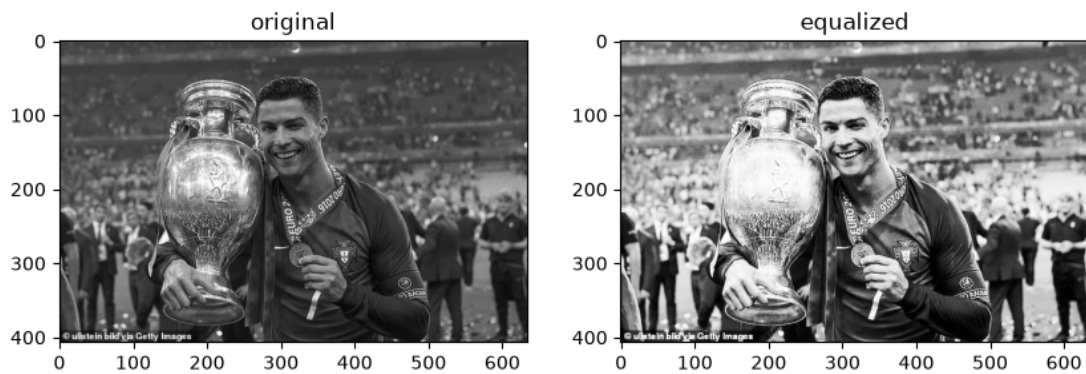


Figure 4: Original gray image (left) and equalized image (right).

2 Experimental Results

The histogram of the equalized image is computed with the same loop as 9a and plotted alongside the original one.

```
histoEq = np.zeros(256, np.int32)
for y in range(h):
    for x in range(w):
        histoEq[equalized[y, x]] += 1

plt.figure(figsize=(8, 5))
plt.bar(range(256), histo, width=1, alpha=0.6, label='original')
plt.bar(range(256), histoEq, width=1, alpha=0.6, label='equalized')
```

```
plt.xlabel("Gray level")
plt.ylabel("Number of pixels")
plt.title("Histogram before and after equalization")
plt.legend()
plt.grid(True)
plt.show()
```

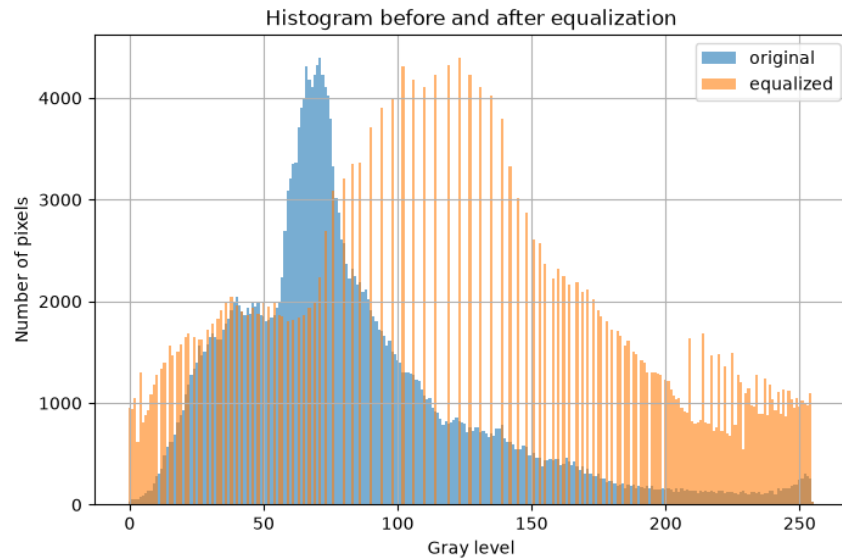


Figure 5: Histogram before and after equalization.

Figure 5 shows that the original histogram has a high peak between 60 and 75, while the equalized histogram is spread over $[0, 255]$.